La Taberna del Bus de Eventos

1. Contexto y Planteamiento del Problema

Imaginen que están desarrollando el sistema central para una popular taberna universitaria llamada "El Gólem Alquímico". En esta taberna, ocurren muchas cosas de forma simultánea:

- Los clientes piden bebidas y comida.
- La cocina prepara los pedidos.
- La barra sirve las bebidas.
- La banda toca música.
- El sistema de sonido ajusta los niveles.
- Un panel de LEDs muestra anuncios y notificaciones.

El dueño, un excéntrico ingeniero retirado, está harto de que sus sistemas estén tan acoplados que, cada vez que quiere agregar una nueva funcionalidad (por ejemplo, que cada vez que la banda toca una canción épica, se disparen efectos de humo), tenga que modificar el código de la banda, del sistema de humo, de la barra... ¡Es un caos!

Su misión, como futuros arquitectos de software, es rediseñar este sistema utilizando un Event Bus para desacoplar todos estos componentes.

2. Objetivos de la Tarea

Al finalizar esta tarea, ustedes deberán ser capaces de:

1. Diseñar una arquitectura basada en eventos para un sistema con múltiples actores.
2. Implementar un Event Bus básico que maneje la publicación y suscripción de eventos.
3. Crear publicadores (Publishers) y suscriptores (Subscribers) independientes que se comuniquen a través del bus.
4. Demostrar el desacoplamiento añadiendo un nuevo suscriptor sin modificar el código de los publicadores existentes.

3. Requisitos Técnicos de la Implementación

Pueden implementar la solución en lenguaje Java. La implementación debe incluir, como mínimo:

A) La Clase `EventBus`:

- Debe tener métodos para:
    - `suscribir(tipo_de_evento, suscriptor)`: Registra una función o objeto para que sea notificado cuando se publique un evento de un tipo específico.
    - `desuscribir(tipo_de_evento, suscriptor)`: Remueve un suscriptor.
    - `publicar(evento)`: Toma un objeto evento y lo notifica a todos los suscriptores registrados para ese tipo de evento.

B) Definición de los Eventos:

- Deben ser objetos/clases inmutables que contengan toda la información relevante.
- Ejemplos de eventos que deberán existir:
    - `PedidoRealizadoEvent` (con data: `mesa_id`, `pedido_id`, `items`)
    - `BebidaServidaEvent` (con data: `mesa_id`, `bebida`)
    - `ComidaPreparadaEvent` (con data: `pedido_id`, `plato`)
    - `BandaTocandoEvent` (con data: `nombre_banda`, `nombre_cancion`, `duracion`)

C) Los Componentes (Suscriptores/Publicadores):

- `SistemaDePedidos` (Publicador): Publica un `PedidoRealizadoEvent`.
- `Barra` (Suscriptor y Publicador): Se suscribe a `PedidoRealizadoEvent` (para ver si hay bebidas). Cuando termina, publica un `BebidaServidaEvent`.
- `Cocina` (Suscriptor y Publicador): Se suscribe a `PedidoRealizadoEvent` (para ver si hay comida). Cuando termina, publica un `ComidaPreparadaEvent`.
- `Banda` (Publicador): Simula tocar una canción cada X segundos y publica un `BandaTocandoEvent`.
- `SistemaDeSonido` (Suscriptor): Se suscribe a `BandaTocandoEvent` y simplemente imprime un mensaje: `"Ajustando ecualizador para: [nombre_cancion]"`.
- `PanelLED` (Suscriptor): Se suscribe a TODO (o a una selección de eventos) e imprime notificaciones coloridas en consola, por ejemplo: `"[PANEL LED] ¡Nueva bebida servida en la Mesa [mesa_id]!"`.

4. Ejemplo de Flujo Deseado

1. El `SistemaDePedidos` recibe un pedido de la "Mesa 5" (1 Cerveza, 1 Hamburguesa). Publica: `PedidoRealizadoEvent(mesa_5, pedido_123, [cerveza, hamburguesa])`.
2. El Event Bus recibe el evento y lo notifica a todos sus suscriptores:
    - `Barra`: Ve que el pedido incluye una cerveza. La prepara y, tras 2 segundos, publica `BebidaServidaEvent(mesa_5, cerveza)`.
    - `Cocina`: Ve que el pedido incluye una hamburguesa. La cocina y, tras 5 segundos, publica `ComidaPreparadaEvent(pedido_123, hamburguesa)`.
3. El Event Bus notifica los nuevos eventos:
    - El `BebidaServidaEvent` es recibido por el `PanelLED`, que muestra un mensaje.
    - El `ComidaPreparadaEvent` es recibido por el `PanelLED`, que muestra otro mensaje.
4. En paralelo, la `Banda` publica un `BandaTocandoEvent("Los Seguidores de Dijkstra", "El Algoritmo del Amor", 180)`. El `SistemaDeSonido` y el `PanelLED` reaccionan en consecuencia.
5. Demostración de Extensibilidad: En el `main` o en una clase aparte, demuestra cómo agregar un nuevo suscriptor `SistemaDeHumo` que reaccione al `BandaTocandoEvent` si la canción es "Through the Fire and Flames". Deben poder hacer esto sin modificar ni un solo carácter en el código de la `Banda` o del `EventBus`. Esto es clave.

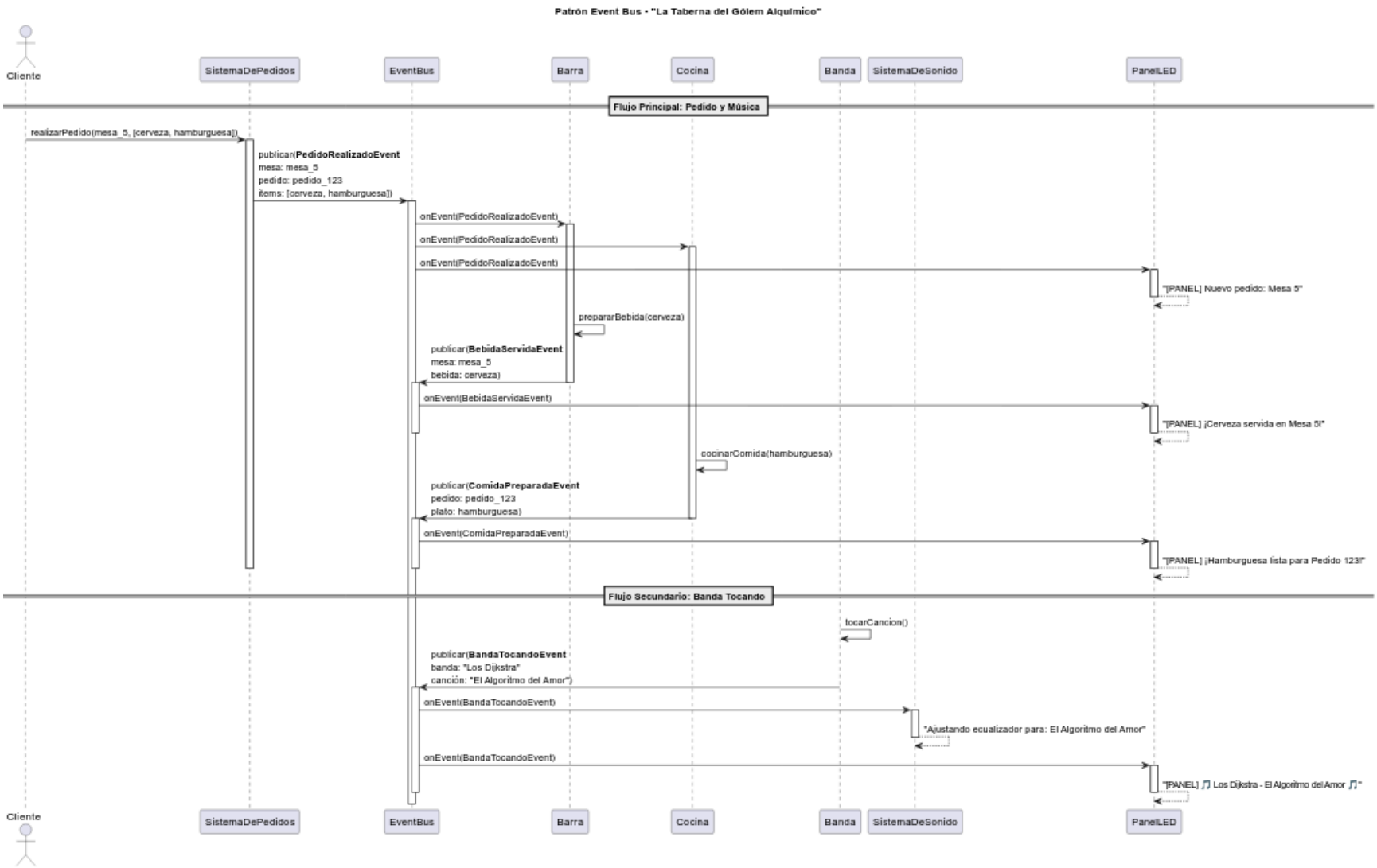
5. Entrega y Criterios de Evaluación

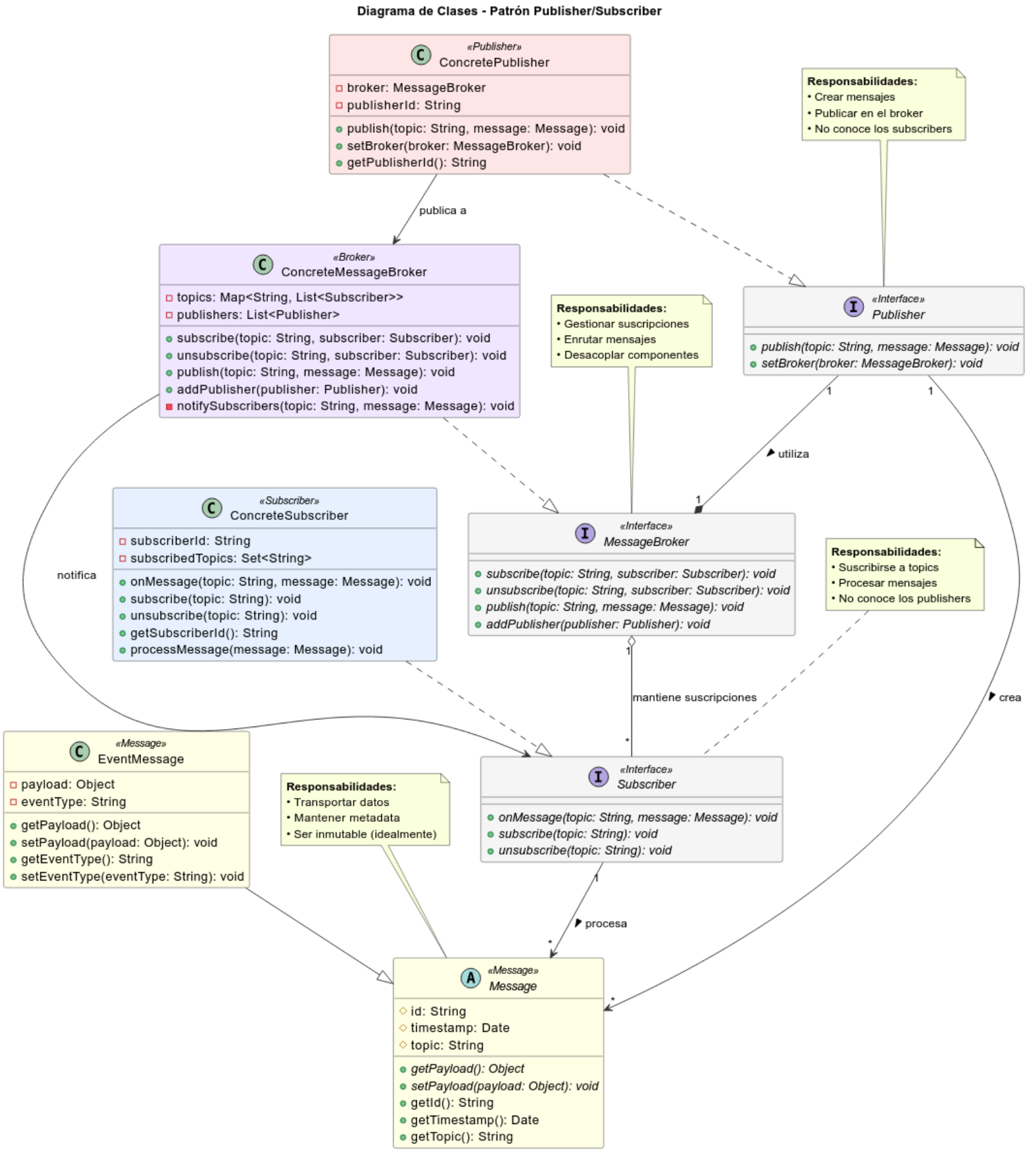
Fecha de Entrega: Una semana a partir de hoy.

Formato de Entrega: Un repositorio de Git (GitHub) con el código fuente y un README.md que explique cómo ejecutar el proyecto.

Documento PDF con Diagramas UML de clases, secuencia y componentes que modelan el software. Usar PlantUML para generar las imágenes e incluir el código.

Interacción un escenario de ejemplo




Diagrama de Clases - Patrón Publisher/Subscriber
«Publisher»
ConcretePublisher
broker: MessageBroker
publisherId: String
publish(topic: String, message: Message): void
setBroker(broker: MessageBroker): void
getPublisherId(): String
Responsabilidades:
• Crear mensajes
• Publicar en el broker
• No conoce los subscribers
publica a
«Broker»
ConcreteMessageBroker
topics: Map<String, List<Subscriber>>
publishers: List<Publisher>
subscribe(topic: String, subscriber: Subscriber): void
unsubscribe(topic: String, subscriber: Subscriber): void
publish(topic: String, message: Message): void
addPublisher(publisher: Publisher): void
notifySubscribers(topic: String, message: Message): void
Responsabilidades:
• Gestionar suscripciones
• Enrutar mensajes
• Desacoplar componentes
«Interface»
Publisher
publish(topic: String, message: Message): void
setBroker(broker: MessageBroker): void
1
1
utiliza
1
«Subscriber»
ConcreteSubscriber
subscriberId: String
subscribedTopics: Set<String>
onMessage(topic: String, message: Message): void
subscribe(topic: String): void
unsubscribe(topic: String): void
getSubscriberId(): String
processMessage(message: Message): void
notifica
«Interface»
MessageBroker
subscribe(topic: String, subscriber: Subscriber): void
unsubscribe(topic: String, subscriber: Subscriber): void
publish(topic: String, message: Message): void
addPublisher(publisher: Publisher): void
Responsabilidades:
• Suscribirse a topics
• Procesar mensajes
• No conoce los publishers
1
mantiene suscripciones
*
crea
«Message»
EventMessage
payload: Object
eventType: String
getPayload(): Object
setPayload(payload: Object): void
getEventType(): String
setEventType(eventType: String): void
Responsabilidades:
• Transportar datos
• Mantener metadata
• Ser inmutable (idealmente)
«Interface»
Subscriber
onMessage(topic: String, message: Message): void
subscribe(topic: String): void
unsubscribe(topic: String): void
1
procesa
*
«Message»
Message
id: String
timestamp: Date
topic: String
getPayload(): Object
setPayload(payload: Object): void
getId(): String
getTimestamp(): Date
getTopic(): String
*